

vue.js
AUG 12

第三章 计算属性与侦听器

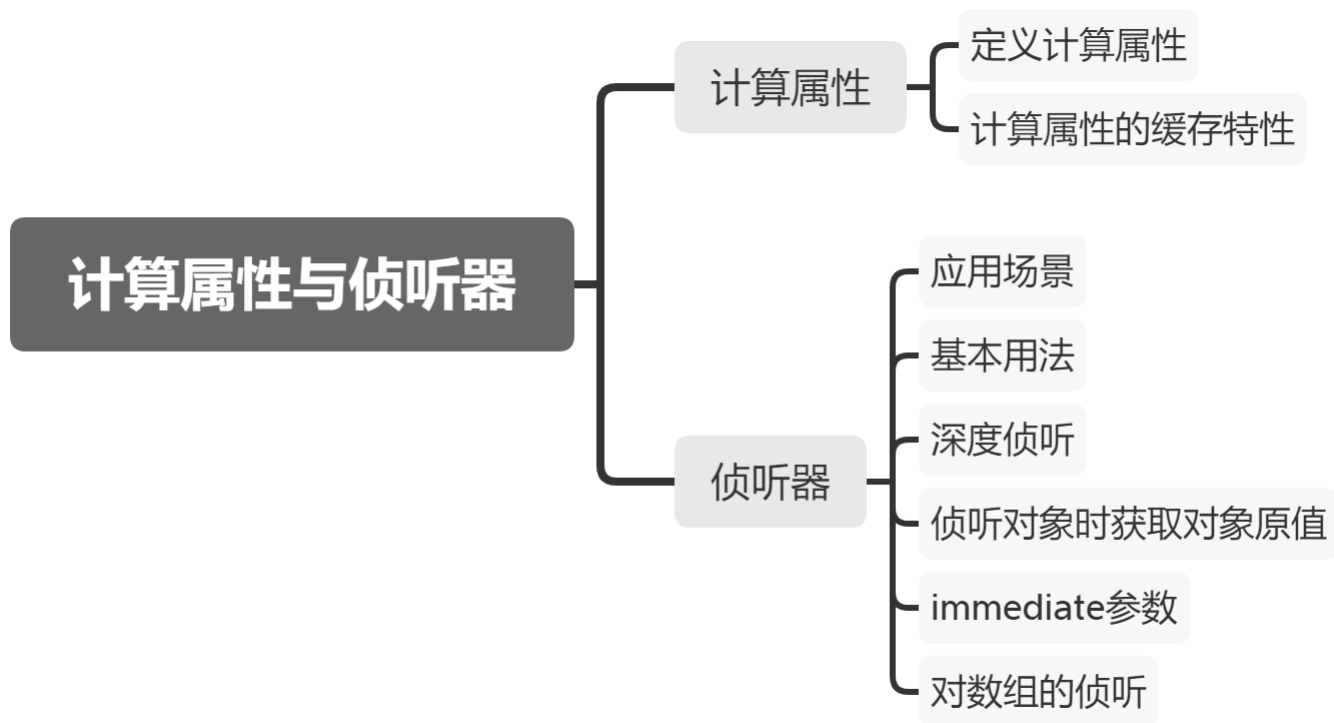
第3章 计算属性与侦听器



- 计算属性

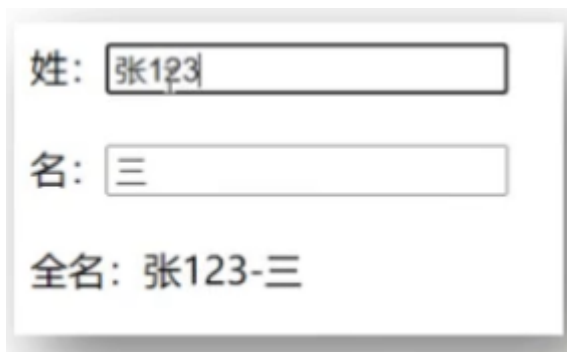
- 侦听属性

第3章 计算属性与侦听器



3.1 计算属性

如果在methods属性中设置一个函数，常被称为“方法”，可以实现对原始数据的加工，从而在视图中使用加工后的数据。Vue.js除了“方法”之外，还有很多其他方式用于处理加工数据，这里介绍一个新的机制，即**computed（计算）属性**。



姓:

名:

全名: 张123-三

```
<div id="root">  
  姓: <input type="text" v-model='firstName'><br/>  
  名: <input type="text" v-model='lastName'><br/>  
  全名: <span>{{firstName}}-{{lastName}}</span>  
</div>
```

```
<script>  
  Vue.config.productionTip = false  
  
  new Vue({  
    el: '#root',  
    data: {  
      firstName: '张',  
      lastName: '三'  
    }  
  })  
</script>
```

3.1 计算属性

```
<div id="root">
  姓: <input type="text" v-model='firstName'><br/>
  名: <input type="text" v-model='lastName'><br/>
  全名: <span>{{firstName}}-{{lastName}}</span>
</div>
<script>
  Vue.config.productionTip = false

  new Vue({
    el: '#root',
    data: {
      firstName: '张',
      lastName: '三'
    }
  })
</script>
```

```
<div id="root">
  姓: <input type="text" v-model='firstName'><br/>
  名: <input type="text" v-model='lastName'><br/>
  全名: <span>{{fullName()}}</span> <br/>
</div>

<script>
  Vue.config.productionTip = false

  new Vue({
    el: '#root',
    data: {
      firstName: '张',
      lastName: '三'
    },
    methods: {
      fullName() {
        console.log(this)
        return this.firstName + '-' + this.lastName
      }
    }
  })
</script>
```

3.1 计算属性

```
<div id="root">
  姓: <input type="text" v-model="firstName"> <br/><br/>
  名: <input type="text" v-model="lastName"> <br/><br/>
  全名: <span>{{fullName}}</span> <br/><br/>
  全名: <span>{{fullName}}</span> <br/><br/>
  全名: <span>{{fullName}}</span> <br/><br/>
  全名: <span>{{fullName}}</span>
</div>
```

```
  全名: <span>{{fullName()}}</span> <br/>
</div>
```

```
<script>
```

```
  Vue.config.productionTip = false
```

```
  new Vue({
    el: '#root',
    data: {
      firstName: '张',
      lastName: '三'
    },
    methods: {
      fullName() {
        console.log(this)
        return this.firstName + '-' + this.lastName
      }
    }
  })
```

```
</script>
```

```
<div id="root">
```

```
  姓: <input type="text" v-model='firstName'><br/>
```

```
  名: <input type="text" v-model='lastName'><br/>
```

```
  全名: <span>{{fullName}}</span>
```

```
</div>
```

```
<script>
```

```
  Vue.config.productionTip = false
```

```
  const vm= new Vue({
```

```
    el: '#root',
```

```
    data: {
```

```
      firstName: '张',
```

```
      lastName: '三'
```

```
    },
```

```
    computed: {
```

```
      fullName: {
```

```
        get() {
```

```
          console.log('get被调用了')
```

```
          return this.firstName + '-' + this.lastName
```

```
        },
```

```
        set(value) {
```

```
          const arr =value.split('-')
```

```
          this.firstName=arr[0]
```

```
          this.lastName=arr[1]
```

```
        }
```

```
      }
```

```
    },
```

```
  })
```

```
</script>
```

3.1 计算属性-简写

```
<script>
Vue.config.productionTip = false

const vm= new Vue({
  el:'#root',
  data:{
    firstName:'张',
    lastName:'三'
  },
  computed:{
    fullName:{
      get(){
        console.log('get被调用了')
        return this.firstName + '-' + this.lastName
      },
      set(value){
        const arr =value.split('-')
        this.firstName=arr[0]
        this.lastName=arr[1]
      }
    }
  }
})
</script>
```

```
//简写
fullName(){
  console.log('get被调用了')
  return this.firstName + '-' + this.lastName
}
```

只读取不修改

■ 定义：

计算属性是“存取器”，本质上是一个函数，但访问（调用）它的时候，像对待一个变量一样，即不带括号，例如上面写作`vm.fullName`，而不是`vm.fullName()`。

3.1 计算属性

1. **定义**：要用的属性不存在，需要通过已有的属性计算得来。
2. **原理**：底层借助了Object.defineProperty方法提供的getter和setter。
3. **get函数什么时候执行**？
 - ① 初次读取的时候会执行一次。
 - ② 当依赖的数据发生改变的时候会再次被调用。
4. **优势**：与method方法相比，内部有**缓存机制**（复用），效率更高，调试方便。

Tips：

- ① 计算属性最终会出现在**VM**上，直接读取使用即可。
- ② 如果计算属性要被修改，那必须写set函数去响应修改，且set中要引起计算时依赖的数据发生改变。

3.2 侦听器

```
<body>
  <div id="root">
    <h2>今天天气很{{info}}</h2>
    <button @click="changeweather">切换天气</button>
  </div>
</body>
```

```
<script>
```

```
Vue.config.productionTip = false
```

```
new Vue({
  el:'#root',
  data:{
    ishot:true
  },
  computed:{
    info(){
      return this.ishot? '炎热':'凉爽'
    }
  },
  methods: {
    changeweather(){
      this.ishot=!this.ishot
    }
  },
})
</script>
```

今天天气很炎热

切换天气

在Vue中，开发者可以自定义方法来进行数据的更新操作，但是不能自动监听数据的状态。如果想在数据更新后进行相应的操作，可以通过侦听器来实现。

3.2 侦听器

侦听器通过watch()函数定义，3个参数

- **侦听器来源**：函数，响应式数据/对象，数组

- **回调函数**

handler(newValue, oldValue)

- **可选参数**

- **deep**：当侦听一个对象时，如果对象中的属性值发生了变化，则无法被监听到。如果想监听到，可以将该选项设为true，表示进行深度监听。默认为False。

- **immediate**：默认情况下，组件在初次加载完毕后不会调用侦听器的回调函数，如果想让侦听器的回调函数立即被调用，则需要将选项设为true。默认为False。

<script>

3.2 侦听属性

```
new Vue({
  el: '#root',
  data: {
    ishot: true
  },
  computed: {
    info() {
      return this.ishot ? '炎热' : '凉爽'
    }
  },
  methods: {
    changeweather() {
      this.ishot = !this.ishot
    }
  },
  watch: {
    ishot: {
      handler(newVal, oldVal) { // ishot发生改变时调用
        console.log('ishot被修改了', newVal, oldVal)
      }
    }
  }
})
</script>
```

侦听属性watch：

1. 当被侦听的属性变化时，回调函数自动调用进行相关工作
2. 监视的属性必须存在，才能进行侦听

3.2 侦听属性

■ 深度侦听

1. Vue中的watch默认不检测对象内部值的变化（一层）
2. 配置deep:true可以监测对象内部值的变化（多层）

Tips：

- ① Vue自身可以监测到对象内部值的改变，但它提供的watch不可以
- ② 使用watch时根据数据具体结构，决定是否使用deep

<h3>a的值是{{numbers.a}}</h3>

<button @click="numbers.a++">点我a+1</button>

```
new Vue({
  el: '#root',
  data: {
    ishot: true,
    numbers: {
      a: 1,
      b: 2
    }
  },
})
```

```
'numbers.a': {
  handler() {
    console.log('a被修改了')
  }
}
```

侦听多级结构中某个属性的变化

```
numbers: {
  deep: true,
  handler() {
    console.log('numbers被修改了')
  }
}
```

侦听多级结构中所有属性的变化

3.2 侦听属性

■ 侦听简写方式

```
watch:{
  isHot:{ I
    // immediate:true, //初始化时让handler调用一下
    // deep:true, //深度监视
    handler(newValue,oldValue){
      console.log('isHot被修改了',newValue,oldValue)
    }
  },
}
```



```
//简写
isHot(newValue,oldValue){
  console.log('isHot被修改了',newValue,oldValue)
}
```

3.2 侦听属性

■ watch与computed

```
new Vue({
  el: '#root',
  data: {
    firstName: '张',
    lastName: '三'
  },
  computed: {
    fullName() {
      console.log(this)
      return this.firstName + '-' + this.lastName
    }
  }
})
```

```
const vm = new Vue({
  el: '#root',
  data: {
    firstName: '张',
    lastName: '三',
    fullName: '张-三'
  },
  watch: {
    firstName(newValue, oldValue) {
      this.fullName = newValue + '-' + this.lastName
    },
    lastName(newValue, oldValue) {
      this.fullName = this.firstName + '-' + newValue
    }
  }
})
```

3.2 侦听属性

```
fullName(){
  console.log('get被调用了')
  setTimeout(() => {
    return this.firstName + '-' + this.lastName
  }, 1000);
}
```

姓:

名:

全名:

■ watch与computed

姓:

名:

全名: 张-三

```
watch:{
  firstName(val){
    setTimeout(() => {
      this.fullName = val + '-' + this.lastName
    },1000);
  },
  lastName(val){
    this.fullName = this.firstName + '-' + val
  }
}
```

1. computed能完成的功能，watch都能完成。
2. watch能完成的功能，computed不一定都能完成，如异步操作。
3. 被vue所管理的函数最好都写成普通函数，这样this才指向VM或组件实例对象

小结

- 计算属性
- 侦听器